COVER STORY

Quantum Computing for First-Time Learners

DAN-ADRIAN GERMAN, *INDIANA UNIVERSITY*, **CHRISTINA SNYDER,** *WESTMINSTER SCHOOLS*,
JOHN M. PHILLIPS, *INDIANA UNIVERSITY*

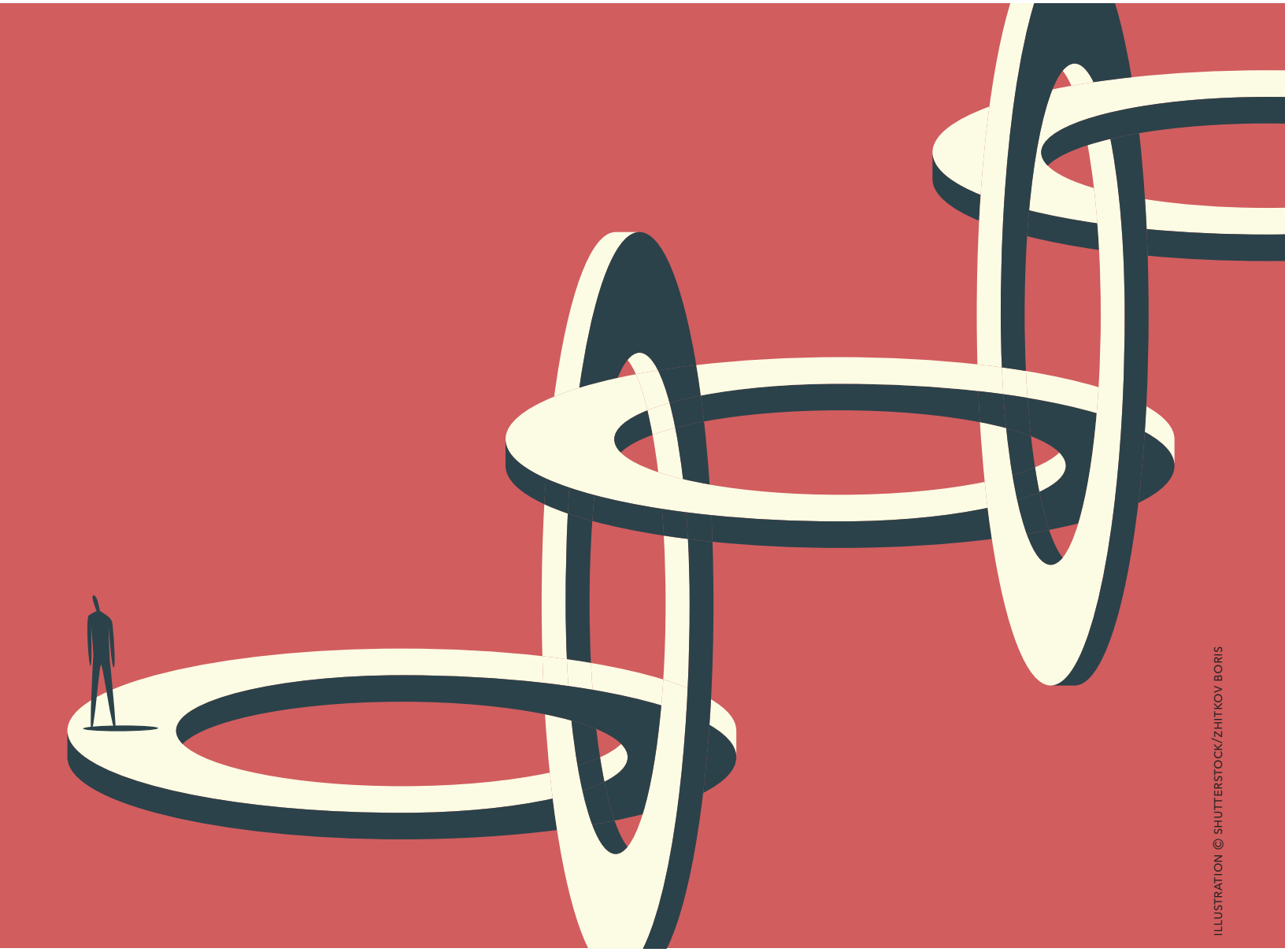


ILLUSTRATION © SHUTTERSTOCK/ZHITKOV BORIS

Experiential Learning

Catching a frisbee is not easy, but dogs and computer science (CS) sophomores seem to be good at it. How they actually do it is still very much subject for debate [6, 7]. That they might be calculating trajectories in real time, using Newton's equations, remains a very unlikely hypothesis. And yet it is undeniable that catching a frisbee demonstrates a working knowledge of physics. How is this knowledge acquired? Recently, the concept of "embodied heuristics" [6] has been proposed as a possible operational explanation. Such a heuristic is a distillation of evolved sensory and motor abilities and is the result of practice and evolution. If you want to be good at catching a frisbee, you need to practice. And if you keep at it, you get good at it. Catching a frisbee is classical physics. Classical physics is all around us. Interaction with it is inevitable, ubiquitous, vital, and can also be fun. Quantum physics, however, is a different story. We cannot directly interact with quantum systems as with classical, so we must build intuition with them in different ways.

Building an Intuition

We emphasize here the need as well as the challenge of building an intuition about quantum phenomena. As Dirac taught us, there is a minimum disturbance that accompanies a measurement, a disturbance that is inherent in the nature of things and can never be improved by experimental technique. "If the disturbance is negligible, then the object is large in an absolute sense, and it can be described by classical physics. [I]f the minimum disturbance accompanying a measurement is nonnegligible, then the object is absolutely small, and its properties fall in the realm of quantum mechanics. The quantum properties of absolutely small particles are not strange; they are just unfamiliar and not subject to our classical intuition." [5] Thus, it may be accurate to say that a quantum object is produced as a particle, propagates as a wave, and is detected as a particle with the probability distribution of a wave. But what difference does that make to a computer scientist? Why do we need quantum mechanics?

Quantum Computing

At the foundation of our field, we have two rewriting systems equivalent in computational power: Turing machines and lambda calculus. Turing machines are important for many reasons, but especially because of two long-held beliefs regarding computation: First, the Church-Turing thesis says everything that is computable can be computed with a Turing machine, although it could sometimes take a very long time. This correctly suggests there are problems that cannot be computed—they are called undecidable problems, the most famous being the halting problem.

Classical physics is all around us. Interaction with it is inevitable, ubiquitous, vital, and can also be fun. Quantum physics, however, is a different story.

Aside from such uncomputable problems, everything else can be computed using a Turing machine.

The extended Church-Turing thesis is the other foundational principle of computer science that says that the performance of all computers is only polynomially faster than a probabilistic Turing machine. In 1993, Bernstein and Vazirani showed that quantum computers could violate the extended Church-Turing thesis. Their quantum algorithm offered an exponential speedup over any classical algorithm for a certain computational task called recursive Fourier sampling. Quantum computation is the only model of computation to date to violate the extended Church-Turing thesis, and therefore only quantum computers are capable of exponential speedups over classical computers. This is because quantum computers harness quantum mechanics to compute by different rules than classical computers do. Although quantum computers don't actually compute faster than classical ones, the rules unique to quantum computing allow new algorithms with new, shorter routes to solutions. It's important to understand that quantum computers would not violate the regular Church-Turing thesis. What is impossible to compute will remain impossible. The hope is that quantum computers will efficiently solve problems that are inefficient on classical computers. One such problem is the factoring of very large numbers. Another one is simulating nature with computers. Nature appears to follow the laws of quantum mechanics. Sometimes classical computers can struggle to crunch the numbers to figure out what nature is doing, but quantum computers play by different rules. They don't need to crunch these numbers per se; they can simply mimic nature rather than approximate it numerically like the classical computers do. That's because, like nature, quantum computers are quantum. The potential here is enormous not just for understanding physics but for designing new materials, such as medicine.

Student Agency

Having decided that the topic is important, we now ask ourselves what learner-sighted teaching technique is best suited here. Education should foster independent exploration and construction of knowledge rather than passive acceptance of instruction, encouraging agency in students. We know from research that students can have more robust

learning experiences when what happens in school is relevant to their lives, helps them connect to a larger purpose, and is grounded in a sense of belonging. For this, the system must be responsive to their goals, interests, and sense of self and community. Student agency, or the ability to manage one's learning, increases motivation and engagement, fosters critical thinking and problem-solving skills, builds autonomy and responsibility, and cultivates a lifelong learning mindset. Allowing students to have a voice and to make choices in their education empowers them to take ownership of their learning journey, leading to deeper understanding, greater confidence, and better preparation for future challenges. Though we agree that a motivated student will always be in pursuit of knowledge, in school we often find knowledge to be in heavy pursuit of the student. Furthermore, we believe all students are intrinsically motivated to learn but become unmotivated if they repeatedly fail. Every student has the basic needs to belong, to be competent, and to influence what happens to them; motivation to learn only exists when these three conditions are satisfied[1]. With this in mind, we have developed an operational approach to jumpstarting quantum computing education in learners as early as middle or high school. Here we restrict ourselves to present the phase kickback phenomenon and the Bernstein-Vazirani algorithm using just the basic rules of arithmetic. Our approach is based on a string-rewriting system invented by Terry Rudolph and introduced in his 2017 book "Q is for Quantum" [10, 11, 12]. We start from classical bits and little by little we introduce phase, superposition, and interference. We show the simple rules that can help a middle school student trace qubits through a quantum circuit. The conventional approach to teaching quantum computation is via linear algebra in a complex vector state. The reality is that, in the U.S., CS students at the college/university level typically lack that mathematical background. This puts CS sophomores in the same category with high school and middle school students: One group already at the university level could have taken the relevant math classes when they had the opportunity but chose not to, and the other group is too young to have had this opportunity to take math classes and build the necessary foundation but might do so in the future. For both groups, the method presented here—using Rudolph's *misty states*—provides a viable shortcut to the heart of the mathematical formalism needed for quantum computation. A Rudolph misty state is essentially an unnormalized quantum state. The terminology here is non-standard; a similar case is the quantum gate introduced as the *PETE box*, for which the widely used terminology is "the Hadamard gate" and which is presented in detail later in this article.

In what follows, we show how to always verify what we do, using the misty-states formalism, with circuits implemented in Qiskit. The reader is invited to read along with a pen and some paper. A laptop would come in handy as well.

Misty States

The 12-year-olds of today may well have access to large quantum computers before they leave their teenage years. Yet, a standard educational trajectory would see them still several years away from learning enough quantum theory to explore this technology's amazing potential meaningfully. In addition to barriers of convention ("This is the order in which things have always been taught") there are math-related barriers ("You can't understand quantum theory until you have mastered linear algebra in a complex vector space"). But, as has been shown, and in true CS spirit, it is possible to replace linear algebra with some string-rewriting rules [12]; it is also important to note that our focus is quantum computing and not quantum mechanics or quantum physics in general. Learning quantum computing is much easier [4] than learning quantum mechanics because quantum computing deals with a simple subset of quantum mechanics, as follows: (a) a qubit—the foundation of quantum computing—is the simplest non-trivial quantum system; (b) you never have to solve the Schrödinger equation or even learn what it is, because the quantum systems that carry out quantum computations evolve in a controlled manner based on the quantum gates applied to them; and (c) there's already a model of quantum computation, so the most difficult aspect of quantum mechanics—the art of applying it to real systems—is absent. We approach presentation from the mindset of maker-centered learning: "What I cannot create I do not understand" is a good description of that persuasion and a quote from Richard Feynman. From here on, in our discussion of any quantum computation concept, we advocate an environment of concrete representations via Python, Qiskit, and the misty-states formalism (the method developed and introduced by Terry Rudolph).

Maker-Centered Learning

According to Piaget, "Children in the early years of primary school need concrete[2] objects, pictures, actions, and symbols to develop mathematical meanings." The same is true of students who lack a certain background or affinity for the pure structures of mathematics. This is where the simplicity of the misty-state formalism shines through. Piaget also said "[l]ogic and mathematics are nothing but

[1] For all the talk that dogs and CS sophomores have short attention spans, the truth is that individuals in both groups typically exhibit laser focus when the motivation (and determination on what to focus) comes from inside.

[2] Our brains need to interact with something in order to create a model of it. As Papert puts it: "You can't think about thinking without thinking about thinking about something."

We approach presentation from the mindset of maker-centered learning: "What I cannot create I cannot understand" is a good description of that persuasion and a quote from Richard Feynman.

specialized linguistic structures." The misty-state formalism can facilitate access to both. Another quote, from Seymour Papert, is relevant here: "My basic idea is that programming is the most powerful medium of developing the sophisticated and rigorous thinking needed for mathematics." [10] So our approach is trying to scaffold the knowledge needed to understand quantum computing and quantum information science starting from computing in Python in a notebook (Google Colab). We build an understanding of the misty-state formalism and then use it to define, recognize, and synthesize (operationally, in Qiskit and Python) the following concepts: superposition, phase, interference, entanglement, quantum gates and quantum circuits, the Deutsch-Josza algorithm, the Grover search algorithm, the Bernstein-Vazirani algorithm (and the phase kickback phenomenon that makes it possible) along with superdense coding and the GHZ game (quantum pseudo-telepathy via quantum entanglement). We then need to extend the system and present quantum teleportation and the phenomenon known as entanglement swapping (i.e., teleportation of entanglement). In this article, we only present the Bernstein-Vazirani algorithm via phase kickback and misty states. The rest has been presented and is available elsewhere and is now essentially part of the CS2023 recommendations and report as a separate knowledge unit (KU).

Quantum Flytrap

We emphasize that our main goal is not quantum mechanics. But we need to stress that "[s]tudents and professionals interested in quantum information sciences need to adopt a different way of thinking[3] than the one used to construct today's (classical) algorithms. This certainly presents tremendous challenges, since for many years, computer science students have been led to believe they can get by with some knowledge of discrete mathematics and little understanding of physics at all. However, of necessity, in quantum computing we are going back to the age when a stronger relationship between physics and computer science existed." [8]

Having said all of this, we also need to point out that we don't consider *detailed* knowledge of quantum mechanics a necessity for a CS student unless they decide to choose a career in building quantum-computing hardware. Here, as is also recommended in the CS2023 KU, we only promote an appreciation of and familiarity with the main quantum concepts: qubit, state, phase, interference, entanglement, teleportation, measurement, sensing, coherence, quantum communication, and the main differences between quantum information science (QIS) and quantum mechanics. An environment facilitating direct interaction with these concepts is the Quantum Flytrap [3], which self-describes as a no-code IDE for quantum computing. Quantum Flytrap was a startup that created no-code tools to make quantum computing accessible to a broader audience by providing visual, interactive simulations of quantum phenomena and experiments and/or to increase public understanding and literacy in quantum technologies through accessible and interactive tools. Although the company closed in 2022, its legacy includes the open source tools it developed, such as the Virtual Lab, which continues to serve as an educational resource for quantum mechanics. We strongly encourage the use of Quantum Flytrap's Virtual Lab in the classroom.

We made it clear that we consider experiential learning a *sine qua non* feature of learning for the kind of learners and topics we have in mind. In such a process, building an intuition via embodied heuristics is fundamental, but direct interaction with the world of the very small is expensive and has to be mitigated since we're so big. John Preskill once remarked: "Perhaps kids who grow up playing quantum games will acquire a visceral understanding of quantum phenomena that our generation lacks."[11] With this in mind, we advocate the use of Quantum Flytrap as a tool to complement the system developed and introduced by Terry Rudolph (i.e., the misty-states formalism), which we will introduce next.

Bernstein-Vazirani

The problem states that we have a circuit in which we have placed a number of (quantum) gates. The circuit will be presented to us as a black box. It will have a number of inputs and an equal number of outputs. We will be asked to determine the internal connectivity of the black box by just interacting with it from the outside. Using only classical physics (gates and principles), we can only conclude that the task of determining what the black box looks like on the inside is linear in the number of inputs. But if we are allowed to use quantum physics (both hardware and principles), the same task can be solved in just one step, regardless of how many inputs the circuit has. Here's an example:

[3] To illustrate the dramatic impact of this new kind of thinking, we highlight the work of Ewin Tang, who in her 2018 undergraduate thesis, supervised by Scott Aaronson, developed classical algorithms matching the performance of the fastest-known quantum algorithms for the matrix completion problem. Inspired by the quantum algorithms of Kerenidis and Prakash, she developed classical analogs of quantum sampling techniques thus "dequantizing" them and exponentially improving over the best-known classical algorithms. Prior to her results, it was widely assumed that no fast classical algorithm existed.

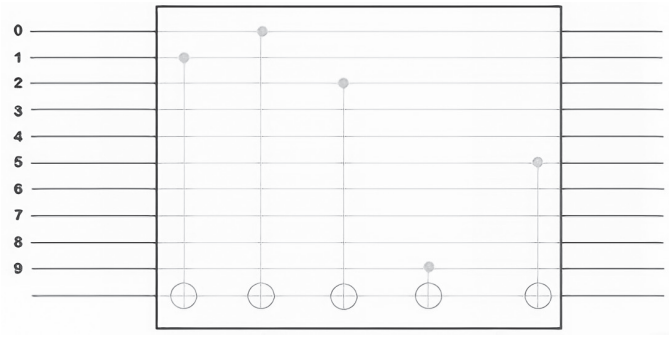


Figure 1. A Bernstein-Vazirani challenge. The black box contains C-NOT gates; they are introduced below.

It is worth mentioning here that the box in Figure 1 is indeed meant to be a black box. The pattern of C-NOT gates is visible here only because we want to show how it can be determined correctly from the outside—so it appears not to be black only for illustrative purposes. We will now define the gates and the formalism we need.

Familiarity with the first part of Rudolph [12] is desirable but won't be assumed. As a result, we first introduce some of the material already in the book (NOT and C-NOT gates along with the Hadamard (PETE) gate). We then prove the phase kickback phenomenon via misty states and use it to solve the Bernstein-Vazirani challenge.

The NOT Gate

An excellent resource here is the 20-minute video [15] available on the book's [13] website.

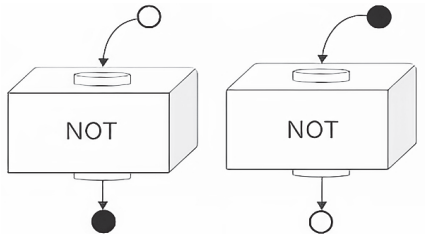


Figure 2. The NOT gate flips its input: NOT(W) = B and NOT(B) = W.

The classical bits are 0 and 1. We can represent them as W and B and draw them as a white or black ball. Indeed, they are classical values. A quantum bit (qubit) is a more complex entity, but when we measure a qubit we only get one of these two values, W or B. The effect of this gate is consistent with our knowledge of the classical NOT gate.

The C-NOT Gate

The controlled-NOT (C-NOT) gate has two inputs: a target and a control. It works by flipping the target, that is, acting as a NOT gate on the target when the control is a black ball. Here's a diagram, redrawn from the book:

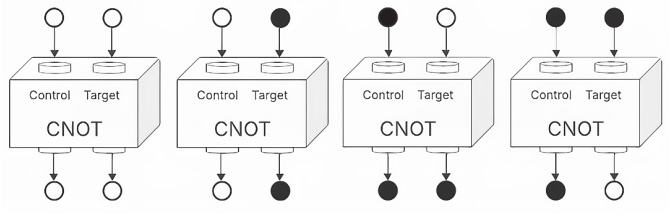


Figure 3. Behavior of the two qubit gate C-NOT.

A Simple Circuit

Next, we consider that by stacking boxes on top of each other, we can use the output of one box as the input to another. For example, we can stack two NOT boxes, as shown on page 8 in Rudolph [12], with the result that the output now matches the input: NOT(NOT(W)) = W and NOT(NOT(B)) = B. We assume familiarity with Python and Google Colab. Naturally, as advocates of the maker-centered learning principles, our next step is to implement this arrangement in Qiskit (the X gate is the quantum NOT):

```
[1] !pip install qiskit pylatexenc

[2] from qiskit import QuantumCircuit

[3] qc = QuantumCircuit(1)
    qc.x(0)
    qc.x(0)
    display(qc.draw('mpl', initial_state=True))
```

```
[4] from qiskit.quantum_info import Statevector
    a = Statevector.from_instruction(qc)
    a.draw('latex')
```

$|0\rangle$

Figure 4. Stacking two NOT boxes in Python (Qiskit) recovers the input, as expected.

Access to the quantum emulator is immediate (e.g., in Google Colab). Access to an actual quantum computer, on the other hand, is not immediate though it's definitely not too prohibitive, either. The good news is that for small examples, like those presented here, an emulator is just as good as the real thing. We now introduce the Hadamard gate.

A Necessary Detour

The Hadamard gate is a fundamental single-qubit quantum gate used to create superposition states. In the book [12], it is known as the PETE box. As Rudolph indicates in the FAQ on the book's website [13], it is the only "actually quantum" box used in the book; all other boxes in the book just shuffle colors around and would be at home in a classical computer.

This group of gates is universal; every calculation can be done (to good-enough accuracy and perhaps with a small overhead) using only PETE boxes and the classical boxes. The reason is a remarkable mathematical result due to Shi, leveraging another powerful result due to Kitaev. "There's a citation at the end of the book," Rudolph states. "A few years ago I was in the middle of pondering this result when I realized I was running late to give a talk at a math camp for 12-14 year olds [...] run in part by my friend Pete Shadbolt. I raced for the tube, and while on it thought about what could I explain to these kids that wasn't the usual jargon-filled quantum fluff. And so here we are."

Our goal in this article is to introduce the misty-state formalism from Rudolph [12] in its pure form using no coefficients whatsoever and use it to prove the phase kick-back phenomenon. With the pure misty-states formalism exactly as defined in the book one can show entanglement, Deutsch-Josza, Grover search, superdense coding, the GHZ game, and entanglement swapping. It is true that one needs to extend this formalism to properly deal with phenomena such as W-entangled states (involving controlled-Hadamard gates and arbitrary rotations) and teleportation (since the input to the quantum teleportation algorithm is an arbitrary[4] quantum state) but at that point the extension feels natural.

The misty formalism is universal in as much as you can use it to do any quantum calculation with only a small overhead. According to Rudolph in his blog, Q is For Quantum, "I should reiterate I am not advocating that we should recast all of quantum theory into this formalism. The misty-state picture is a good way of getting people to the heart of some nontrivial quantum theory without them having to absorb a boatload of irrelevant math. But that math is not largely irrelevant if you actually want to work in the field, it makes many things much easier." [13] Math is our ultimate goal here as well. For example, we'd like our readers to be able and ready to read [1, 9, 14] as soon as they master the contents of our class.

We already have the NOT and C-NOT gates. We now define the Hadamard gate (also known as PETE box).

The PETE Box

We capture the behavior shown in Figure 5 by introducing the superposition operator. As a drawing, it is represented as a cloud; hence, the name "misty state" that is used for a superposition state. This, again, would be a great opportunity to watch (or rewatch) Terry's video [13] from the book's website at [13]. In text, we can use the following two representations corresponding to each of the situations shown above: H(W) = [W, B] and H(B) = [W, -B]. The notation says, in essence, that there are two outcomes, and each one is equally likely to be measured. Here's how we represent these two transformations graphically:

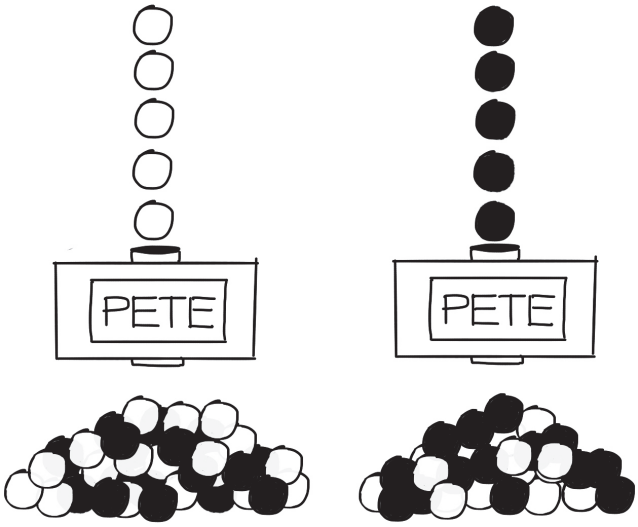


Figure 5. Behavior of the Hadamard gate (also known as the PETE box in Rudolph [12]).

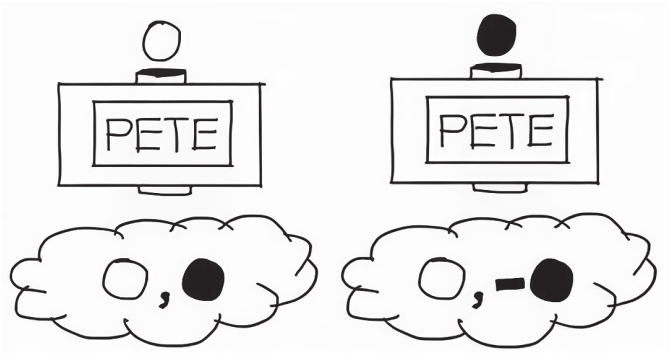


Figure 6. Misty states are superposition states (as shown in Rudolph [12]). A negative sign (phase) shows on the right.

The Z Gate

The Z gate is introduced here as an exercise. Its definition is Z(W) = W and Z(B) = −B. Show that, just like for NOT, stacking two Z boxes leaves the input unchanged. We will soon learn this is a general property of quantum gates, and our next goal will be to prove it for PETE boxes (or Hadamard gates). That H(H(W)) = W and H(H(B)) = B is both non-trivial and very instructive. We can also demonstrate that experimentally in the Quantum Flytrap.

Exercise. Show that Z(H(W)) = H(X(W)) and Z(H(B)) = H(X(B)).

Linearity of Quantum Operators

A misty state, so far, is just a sum of two states with probability amplitudes equal to each other. The phase we encountered thus far is just a multiplication with the scalar −1. In quantum mechanics, linearity of operators means they satisfy two key properties: (a) they preserve the sum of states, and (b) they preserve scalar multiplication. This

property is fundamental to the superposition principle and how quantum states evolve over time. Therefore, we shall enforce it here. As a result, we have the following diagram (Figure 7), whic shows how a NOT gate acts on a superposition of states:

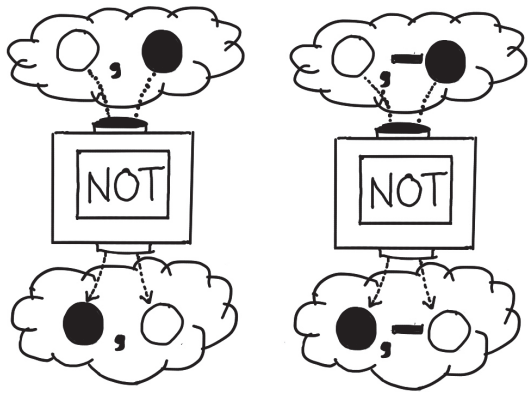


Figure 7. The effect of NOT gate on superposition of states. This diagram reproduced by permission from Rudolph [12].

We can describe what happens in Figure 7 as follows:

NOT([W, B]) = [NOT(W), NOT(B)] = [B, W] = [W, B]

We take the opportunity to point out that as in a sum, the order of factors (i.e., of states in a superposition operator) does not matter, so the NOT gate in effect leaves the first misty state unchanged. In the case of the second diagram, we have: NOT([W, −B]) = [NOT(W), NOT(−B)] = [B, −NOT(B)] = [B, −W] = −[−B, W] = −[W, -B]

We have in fact proved that these are the two eigenvectors (CS: fixed points) of the NOT gate. In the process, we illustrated linearity of phase and superposition operators with respect to the NOT gate. By a similar process, we show how stacking two PETE boxes (or as everybody else knows[5] them, Hadamard gates) leaves the input unchanged [5].

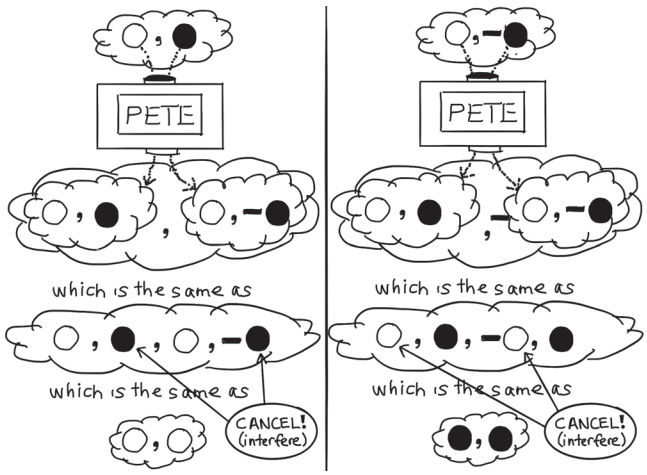


Figure 8. The effect of the PETE box (Hadamard gate) on two superpositions of states. Also from Rudolph [12].

This part further uses the fact that a superposition operator is a sum and under certain conditions (i.e., when the superpositions are at the same depth and have the same number of distinct states), we can combine two mists by fading their boundaries so they can combine (join together) into a larger mist. Here's how this happens (Figure 8) in the notation we used to restate what was going on in Figure 7:

H(H(W)) = H([W, B]) = [H(W), H(B)] = [[W, B], [W, −B]] = [W, B, W, −B] = [W, W] = W

Now please look at Figure 8 as it shows (diagrammatically) what we wrote above and below:

H(H(B)) = H([W, −B]) = [H(W), −H(B)] = [[W, B], [−W, B]] = [W, B, −W, B] = [B, B] = B

These relationships can be easily represented, reproduced, and confirmed as experimental setups in Quantum Flytrap.

Systems of Two Qubits

There are four possible combinations of two qubits: WW, WB, BW, and BB. We can represent this with white and black balls (or blobs), and we say that while they resemble multiplication they lack one important property of multiplication as they are not commutative. Thus, WB and BW are different, so order matters; other than that, we can carry over some of the other properties encountered in multiplication: B [W, B], for example, is the same as [BW, BB]. This, in effect, is how we define entanglement. Two (or more) particles are entangled when they are all described by the same wave function. For us, this means the expression that represents the state of the two (or more) qubits can't be separated as a product of factors each representing an individual qubit. Thus, because [BW, BB] = B [W, B] this equation does not describe a system of two entangled qubits. However, a state like [BW, WB] cannot be split into a product of two states and thus represents an entangled state of two qubits (it's one of the Bell states). There is no entanglement in the Bernstein-Vazirani challenge that we discuss, but we will be working with systems of two qubits, so we wanted to clarify this up front. The phase kickback is this situation (that is, this is what we need to prove):

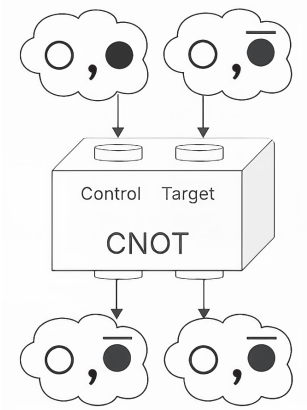


Figure 9. The phase of the target qubit is being "kicked back" to the first qubit. Target influences control.

We started with frisbees and classical mechanics and extolled the virtues of experiential learning.

How do we prove that's what is happening here? Let's start by writing the input as a system of two qubits. It is convenient here to keep the second qubit as a superposition and work with it as such. We have by distributivity:

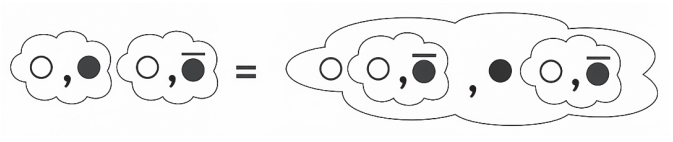


Figure 10. The input [W, B] [W, −B] = [W [W, −B] , B [W, −B]] in diagrammatic form.

Now that we have the input expressed as such, let's pass it through the C-NOT gate and transform it using the rules of engagement already mentioned for this gate. In Figure 11, each pair of qubits passes through the C-NOT gate. The first one is placing a W on the control, which means the gate will leave the second qubit unchanged. The second pair has a B on the control, which flips the second qubit. The rule for flipping a superposition of states (via NOT) has been shown before and appears in Figure 12.

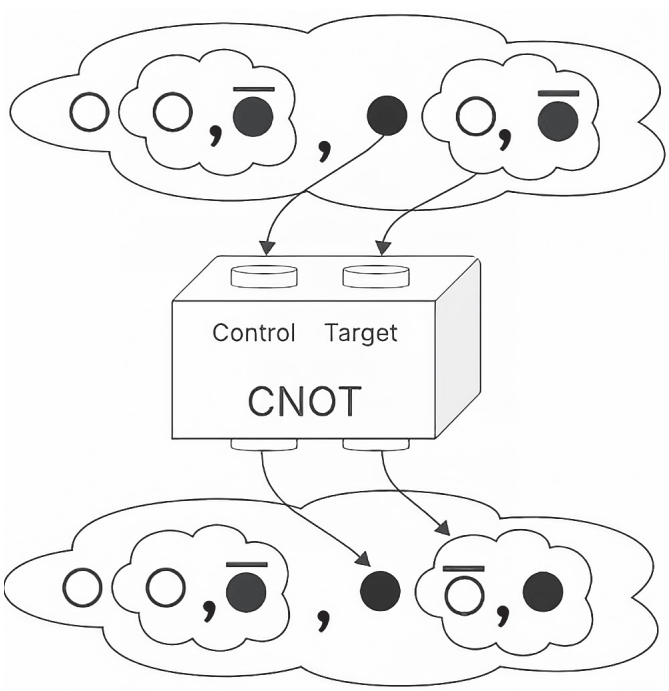


Figure 11. Effect of the C-NOT gate on our two qubit input.

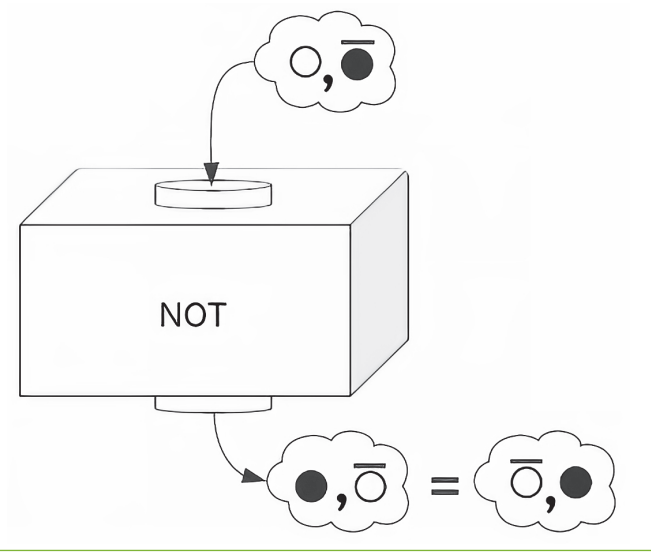


Figure 12. The effect of the NOT gate on a superposition of states. See also right side of Figure 7.

The purpose of Figure 12 is to support the transformation shown in Figure 11. Thus, some readers might consider the picture to be redundant, while some might prefer to use shorthand to describe it, for example, NOT([W, −B]) = [−W, B]. We covered this earlier when we said that the input here is one of the two eigenvectors of the NOT gate. Since the superposition operator is actually a sum, as we said before, the order of states in a mist is not important, but an order is usually preferred and the phase distributes over the constituent states, as shown in Figure 13.

Figure 13. A negative phase applied to a mist distributes over its constituent states.

Now we can rewrite the second state in the output of Figure 11 as follows: B [−W, B] = B (−[W, -B]) = (−B) [W, −B]

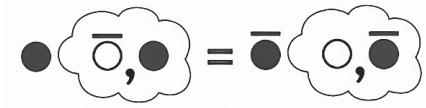


Figure 14. Moving the sign (phase) from the second qubit to the first has this effect.

Figure 15. The output from C-NOT (factoring via reverse FOIL) as a product of states, each one a superposition.

So now we have proved the following:

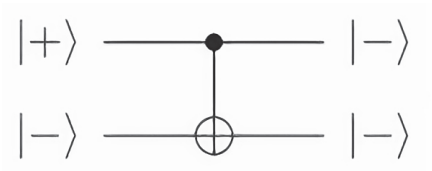


Figure 16. Phase kickback, conventional notation.

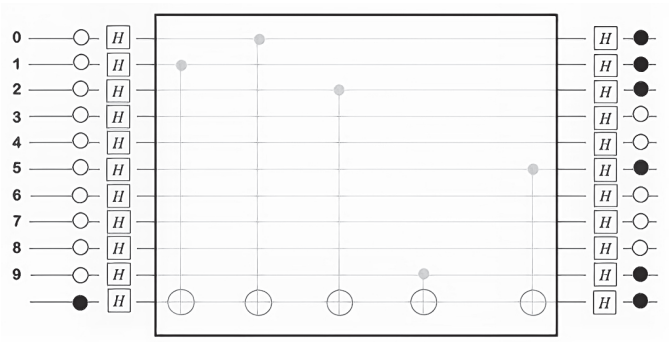


Figure 17. Bernstein-Vazirani challenge: the solution. If the C-NOT gates inside the black box had been oriented the other way, the solution would have been immediate. But since they're as shown above, one would need to test every input in part, thus establishing a linear lower bound for the complexity of finding the pattern. We trade space and hardware for speed. With the previous result and a corresponding number of Hadamard (PETE) boxes, we can determine the structure of the black box in one step.

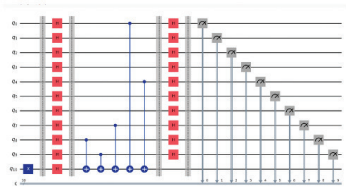
Below we show how this challenge can be implemented in Qiskit:

```
[1] !pip install qiskit qiskit-aer

[2] import qiskit

[3] from qiskit import *
    from qiskit_aer import AerSimulator

[4] secretnumber = '1110010001'
    indices = [1, 0, 2, 9, 5]
    n = len(secretnumber)
    circuit = QuantumCircuit(n+1, n)
    circuit.x(n)
    circuit.barrier()
    circuit.h(range(n))
    circuit.h(n)
    circuit.barrier()
    for index in indices:
      circuit.cx(n - index - 1, n)
    circuit.barrier()
    circuit.h(range(n))
    circuit.barrier()
    circuit.measure(range(n), range(n))
    circuit.draw(output='mpl')
```



```
[5] from qiskit_aer import AerSimulator
    device = AerSimulator()
    shots = 1
    job = device.run(circuit, shots = shots)
    result = job.result()
    counts = result.get_counts(circuit)

[6] print(counts)

    {'1110010001': 1}
```

Figure 18. Creating and measuring the quantum circuit for the Bernstein-Vazirani.

From the previous figure, we observe that the order of inputs is reversed in Qiskit, so the circuit is reflected. As an example, the third C-NOT in the black box in our drawing connects the bottom line, its target, to the third line from the top, the control. In the Qiskit circuit, it connects the bottom line or target with the third line from the bottom; Qiskit numbers the qubit lines in reverse order. Also, the left-to-right order of gates in the black box is not important, but the vertical order of the lines in the input and output is; the output is determined, as predicted, in one shot.

Conclusion

CS2023 makes some excellent recommendations for the first time ever on how to include a knowledge unit on quantum information science, computing, and quantum algorithms. Their proposal is organized in three stages and comprises a short eight-week class; a one-semester class; and a longer, two-semester sequence that, at least in principle, makes heavy use of a lab or fab, depending on resources, in quantum hardware, gates, and circuits. Following those recommendations, we have described our approach of implementing the eight-week syllabus with extended material from Terry Rudolph's groundbreaking "Q is for Quantum" [12]. This material has been tested in the classroom, in various conferences in workshops and tutorials, and at many levels, including creating a faculty learning community (FLC) for high school and middle school CS teachers in the state of Indiana last summer with significant support from the Indiana Computer Science Teachers' Association (CSTA).

Afterword

We started with frisbees and classical mechanics and extolled the virtues of experiential learning. We then discussed what absolutely small means (according to Dirac) and how direct interaction with the quantum world does not come easy. Turing machines and lambda calculus were invented by Alonzo Church and his student Alan Turing; they are the cornerstones of our field and were introduced in the first part of the 1930s, so they are younger than quantum mechanics. Quantum computing is the only model of computation that breaks the strong extended Church-Turing thesis. The misty-state formalism favors learner-sighted teaching techniques, and the Quantum Flytrap along with Python coding with Qiskit in Google Colab notebooks supports maker-centered learning. In this article, we solve a Bernstein-Vazirani challenge using the pure misty-state formalism introduced in Rudolph [12]. We only need the NOT gate, the two-qubit C-NOT gate, the mysterious PETE box, also known as the Hadamard gate, and the concept of superposition. Systems of two or more qubits can exhibit entanglement, but this phenomenon is not present in the Bernstein-Vazirani challenge. The Z gate is introduced as a means to practice with the new system. Linearity of quantum operators translates into very simple operations with B and W, black and white balls. The system devised by Professor Rudolph is incredibly effective and works even with states that break the representation chosen in the book. We would have loved to show you irreducible misty states—for example, the two Hadamard

In this article, we solve a Bernstein-Vazirani challenge using the pure misty state formalism introduced in Rudolph.

eigenvectors have this property—and how useful they are in facilitating abstraction and representation of non-classicality. More exercises would have asked you to design a one-qubit circuit that generates the eigenvectors of the Hadamard gate and then a two-qubit circuit that would do the same. Unfortunately, space restrictions did not allow us to share those with you here. An appendix to this article with those topics that could not make it in the printed version can be found online [15]. ◆

ACKNOWLEDGMENTS
This work was done as part of ACM/IEEE-CS/AAAI Computer Science Curricula 2023 [2]. The authors extend their thanks to Professor Terry Rudolph [17] for allowing the use of the diagrams in his book.

REFERENCES

1. Burd, B. *Quantum Computing Algorithms: Discover How a Little Math Goes a Long Way*. Birmingham, U.K. Packt Publishing (2023).
2. CS2023: ACM/IEEE-CS/AAAI Computer Science Curricula; https://csed.acm.org/
3. Ekert, A. (adviser) Quantum Flytrap: A no-code IDE for quantum computing; https://quantumflytrap.com/
4. Ellwanger, D. *A Quantum Computing Pamphlet*. (2019).
5. Fayer, M.D. *Absolutely Small: How Quantum Theory Explains Our Everyday World*. Farnham, U.K., Amacom Books, Special Edition (2020).
6. Gigerenzer, G. Embodied heuristics. *Frontiers in Psychology* (2021); https://tinyurl.com/embodied-heuristics
7. Haldane, A.G. and Madouros, V. The dog and the frisbee. In Proceedings of the *Fed. Reserve Bank of Kansas City's Annual Economic Policy Symp*. (2012); https://tinyurl.com/the-dog-and-the-frisbee
8. Marinescu, D.C. and Marinescu, G.M. *Approaching Quantum Computing*. London, U.K.: Pearson Education, Inc. (2004).
9. Nielsen, M.A. and Chuang, I.L. Quantum Computation and Quantum Information. Cambridge, U.K., Cambridge University Press (2011)
10. Papert, S. *Mindstorms: Children, Computers, and Powerful Ideas*. Basic Books (1980).
11. Preskill, J. Quantum computing in the NISQ era and beyond. *Quantum 2*, 79 (2018).
12. Rudolph, T. *Q is for Quantum* (2017).
13. Rudolph, T. Book website: https://www.qisforquantum.org
14. Rudolph, T. https://tinyurl.com/terry-rudolph-misty-states
15. Supplemental material; https://homes.luddy.indiana.edu/dgerman/misty-states.html
16. Watrous, J. Understanding Quantum Information and Computation (IBM Quantum Education Series (2025); https://tinyurl.com/watrous
17. Wikipedia, https://en.wikipedia.org/wiki/Terry_Rudolph

DAN-ADRIAN GERMAN
Indiana University – Computer Science, Bloomington, Indiana
dgerman@iu.edu

CHRISTINA SNYDER
Westminster Schools, Atlanta, Georgia
christinasnyder@westminster.net

JOHN M. PHILLIPS
Indiana University – Computer Science, Bloomington, Indiana
phillip2@iu.edu

© 2025 Copyright held by owner/author(s).
DOI: 10.1145/3774781

This work is licensed under Creative Commons Attribution International 4.0.

Association for Computing Machinery

Career & Job Center

The #1 Career Destination to Find Computing Jobs.

Connecting you with top industry employers.

Check out these new features to help you find your next computing job.

- Access to new and exclusive career resources, articles, job searching tips and tools.
- Gain insights and detailed data on the computing industry, including salary, job outlook, 'day in the life' videos, education, and more with our new Career Insights.
- Receive the latest jobs delivered straight to your inbox with **new exclusive Job Flash™ emails**.
- Get a free resume review from an expert writer listing your strengths, weaknesses, and suggestions to give you the best chance of landing an interview.
- Receive an alert every time a job becomes available that matches your personal profile, skills, interests, and preferred location(s).

Visit https://jobs.acm.org/